8.1 When and how should you test the UI?

Web tests have some significant advantages over other types of testing:

- The visual results, when reported well, are an effective way to describe how the user interacts with the application to achieve particular tasks.
- They reproduce end-user behavior much more closely than under-the-hood tests.
- They're a great way to demo features to stakeholders.
- They can significantly reduce the need for manual UI testing, which represents a significant overhead for testers.

A web test, by definition, is designed to verify UI behavior. But web tests, as end-to-end tests, can also be an effective way to illustrate and check how all of the components in the system work together. Used as living documentation, a web test also often does a great job of documenting how a user will use the system to achieve a particular goal. Web tests can also help give business analysts, testers, and stakeholders more confidence in the automated acceptance tests.

8.1.1 The risks of too many web tests

But web tests aren't without a degree of risk. Sometimes when teams start out with BDD, they try to automate all of their requirements exhaustively, almost exclusively using detailed, fine-grained, script-like automated web tests. This is a natural tendency for many testers who come from a background of automated functional testing using commercial tools such as HP's QuickTest Professional. But down this path danger lies. It's very easy to build up a large suite of brittle automated web tests that are costly to maintain and hard to keep up to date. Poorly designed script-like automated web tests often contain a large amount of duplicated code. When a web page is modified, each test that manipulates the page needs to be updated individually, which is time-consuming and error-prone.

Automated web tests generally run much more slowly than non-web tests, and they tend to be more fickle. Some tests may not behave in the same manner from one browser to another, and they may require browser-specific tweaks. Tests can also fail for reasons beyond the control of your code: an incorrect version of Firefox running on the build server, a page timeout because of network latency, and so on.

Record-Replay style scripting tools also present their own particular category of problems. These tools, which include products like Selenium IDE and QuickTest Professional, allow users to record test scripts through a visual tool and replay the tests afterwards. This approach sounds simple and intuitive, but it's deeply flawed. One problem is that these scripts are extremely brittle, hard to read, and unclear about their intent, and are, as a result, virtually unmaintainable. The other problem is that these scripts can't be written until the application development has been completed, which means that they tend to be written as test scripts to verify the implemented behavior, rather than as automated acceptance criteria that contribute to and help guide development efforts from the early stages of the project. For these